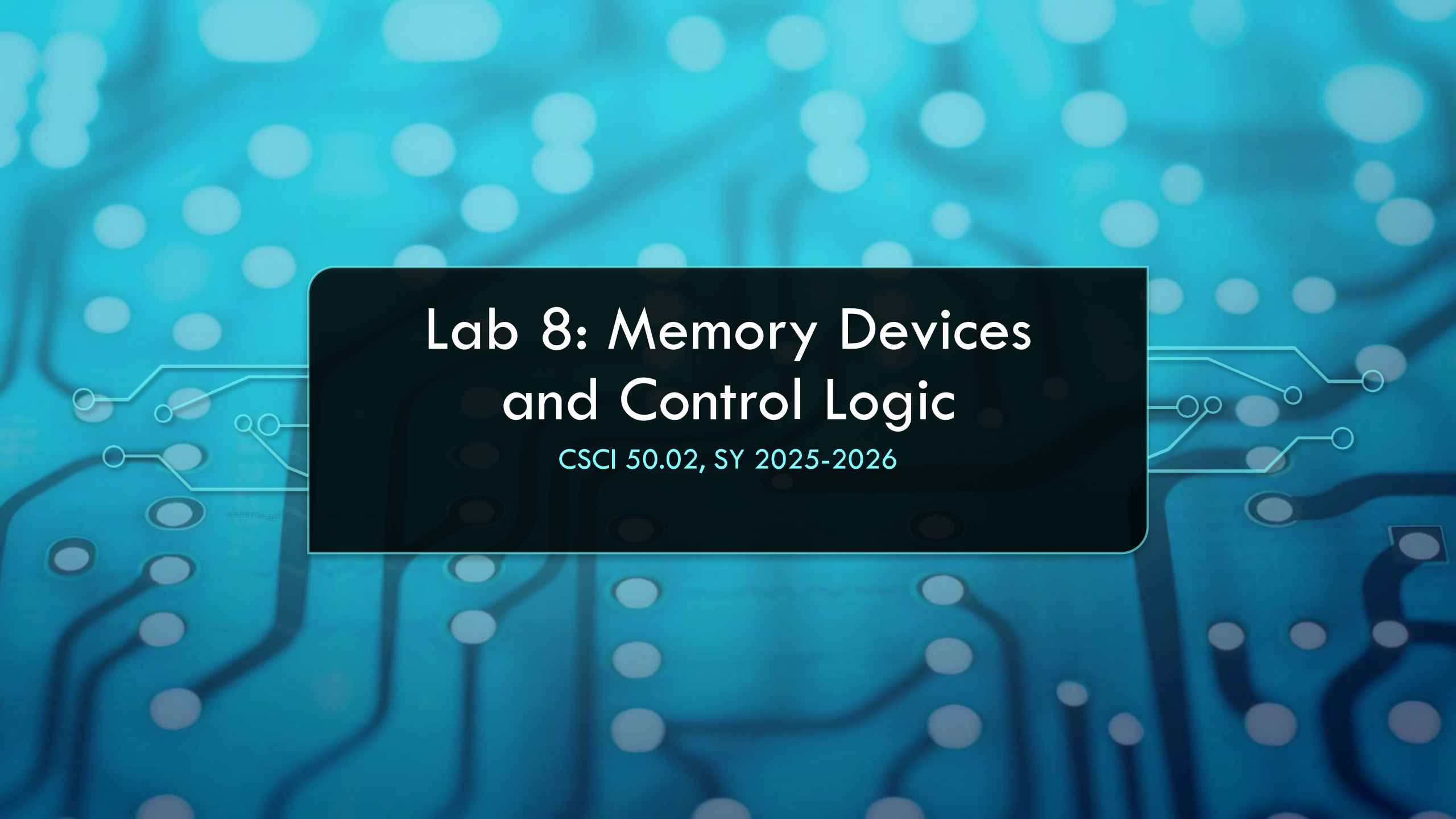




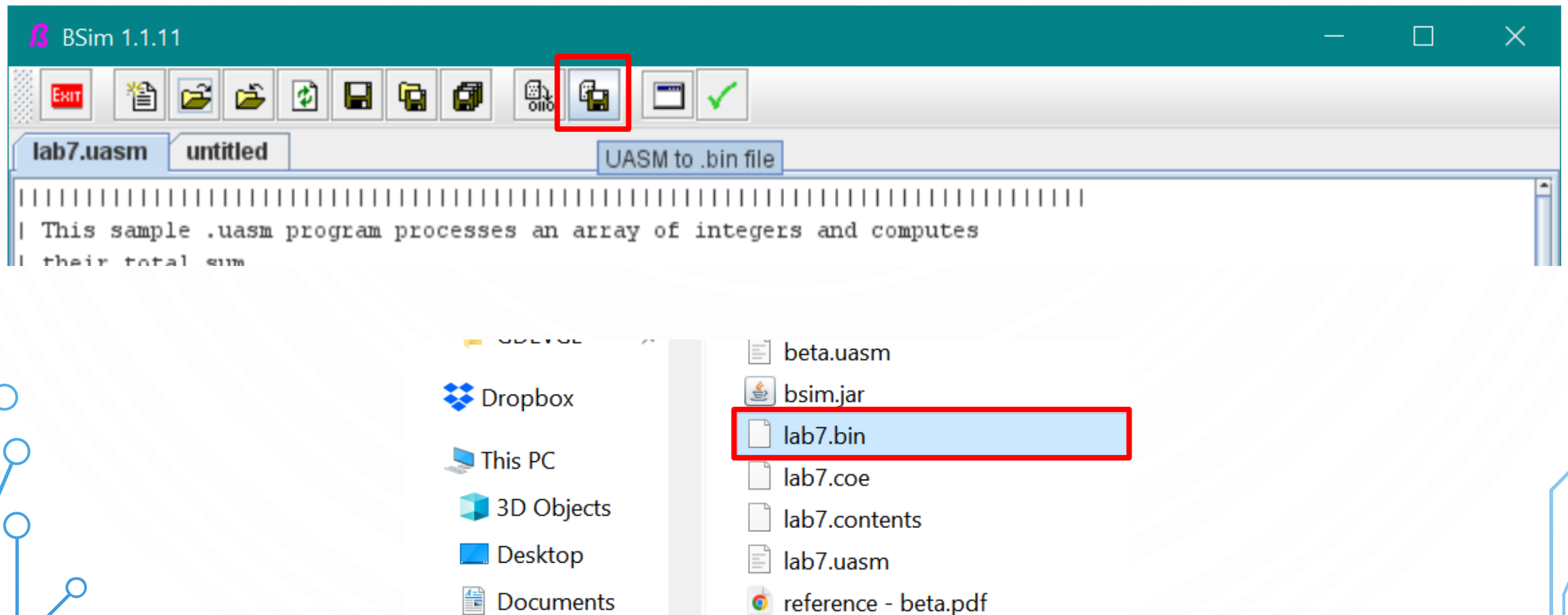
Lab 8: Memory Devices and Control Logic

CSCI 50.02, SY 2025-2026

Before we begin...

- Download lab8.jsim from Canvas
- Create a .bin file of one of your Lab 7 programs (you may pick any of the 3 you made) and rename it to lab7.bin
- Put lab8.jsim and lab7.bin in your JSim folder (and run lab8.jsim in JSim)

How to create a .bin file in BSim



What you should expect when running lab8.jsim (with lab7.bin) in JSim

pc[8:0]	0x000	0x004	0x008	0x00C	0x010	0x014	0x018	0x01C
betaop(opcode[5:0])	LD	ADDC	LD	ADD	SUBC	BNE	ST	???
d(ra[4:0])	31	1	1	3	0	0	31	0
d(rb[4:0])	0	0	0	2	0	31	0	0
d(rc[4:0])	0	1	2	3	0	31	3	0
sd(c[15:0])	412	4	412	4096	1	-5	436	0

- Note that rb and c share the same bits in the instruction – if rb is valid, c is garbage (and vice versa)
- In this example, c is valid (rb is garbage)

INSTRUCTIONS (SUPERVISOR MODE)	
000:601F019C	LD (R31,412,R0)
004:C0210004	LOOP: ADDC (R1,4,R1)
008:6041019C	LD (R1,412,R2)
00C:80631000	ADD (R3,R2,R3)
010:C4000001	SUBC (R0,1,R0)
014:7BE0FFFB	BNE (R0,LOOP)
018:647F01B4	ST (R3,436,R31)
01C:00000000	HALT()

What you should expect when running lab8.jsim (with lab7.bin) in JSim

pc[8:0]	H: 2ns/div	0x000	0x004	0x008	0x00C	0x010	0x014	0x018	0x01C
betaop(opcode[5:0])	H: 2ns/div	LD	ADDC	LD	ADD	SUBC	BNE	ST	???
d(ra[4:0])	H: 2ns/div	31	1	1	3	0	0	31	0
d(rb[4:0])	H: 2ns/div	0	0	0	2	0	31	0	0
d(rc[4:0])	H: 2ns/div	0	1	2	3	0	31	3	0
sd(c[15:0])	H: 2ns/div	412	4	412	4096	1	-5	436	0

- For this ADDC instruction, c is valid (rb is garbage)

INSTRUCTIONS (SUPERVISOR MODE)	
000:601F019C	LD (R31,412,R0)
004:C0210004	LOOP: ADDC (R1,4,R1)
008:6041019C	LD (R1,412,R2)
00C:80631000	ADD (R3,R2,R3)
010:C4000001	SUBC (R0,1,R0)
014:7BE0FFFB	BNE (R0,LOOP)
018:647F01B4	ST (R3,436,R31)
01C:00000000	HALT()

What you should expect when running lab8.jsim (with lab7.bin) in JSim

pc[8:0]	H: 2ns/div	0x000	0x004	0x008	0x00C	0x010	0x014	0x018	0x01C
betaop(opcode[5:0])	H: 2ns/div	LD	ADDC	LD	ADD	SUBC	BNE	ST	???
d(ra[4:0])	H: 2ns/div	31	1	1	3	0	0	31	0
d(rb[4:0])	H: 2ns/div	0	0	0	2	0	31	0	0
d(rc[4:0])	H: 2ns/div	0	1	2	3	0	31	3	0
sd(c[15:0])	H: 2ns/div	412	4	412	4096	1	-5	436	0

INSTRUCTIONS (SUPERVISOR MODE)	
000:601F019C	LD (R31,412,R0)
004:C0210004	LOOP: ADDC (R1,4,R1)
008:6041019C	LD (R1,412,R2)
00C:80631000	ADD (R3,R2,R3)
010:C4000001	SUBC (R0,1,R0)
014:7BE0FFFB	BNE (R0,LOOP)
018:647F01B4	ST (R3,436,R31)
01C:00000000	HALT()

- For this ADD instruction, rb is valid (c is garbage)

What you should expect when running lab8.jsim (with lab7.bin) in JSim

pc[8:0]	H: 2ns/div	0x000	0x004	0x008	0x00C	0x010	0x014	0x018	0x01C
betaop(opcode[5:0])	H: 2ns/div	LD	ADDC	LD	ADD	SUBC	BNE	ST	???
d(ra[4:0])	H: 2ns/div	31	1	1	3	0	0	31	0
d(rb[4:0])	H: 2ns/div	0	0	0	2	0	31	0	0
d(rc[4:0])	H: 2ns/div	0	1	2	3	0	31	3	0
sd(c[15:0])	H: 2ns/div	412	4	412	4096	1	-5	436	0

- This BNE instruction has an implicit rc of r31 (why?)
- What does c = -5 represent?

INSTRUCTIONS (SUPERVISOR MODE)	
000:601F019C	LD(R31,412,R0)
004:C0210004	LOOP: ADDC(R1,4,R1)
008:6041019C	LD(R1,412,R2)
00C:80631000	ADD(R3,R2,R3)
010:C4000001	SUBC(R0,1,R0)
014:7BE0FFFB	BNE(R0,LOOP)
018:647F01B4	ST(R3,436,R31)
01C:00000000	HALT()

What you should expect when running lab8.jsim (with lab7.bin) in JSim

pc[8:0]	H: 2ns/div	0x000	0x004	0x008	0x00C	0x010	0x014	0x018	0x01C
betaop(opcode[5:0])	H: 2ns/div	LD	ADDC	LD	ADD	SUBC	BNE	ST	???
d(ra[4:0])	H: 2ns/div	31	1	1	3	0	0	31	0
d(rb[4:0])	H: 2ns/div	0	0	0	2	0	31	0	0
d(rc[4:0])	H: 2ns/div	0	1	2	3	0	31	3	0
sd(c[15:0])	H: 2ns/div	412	4	412	4096	1	-5	436	0

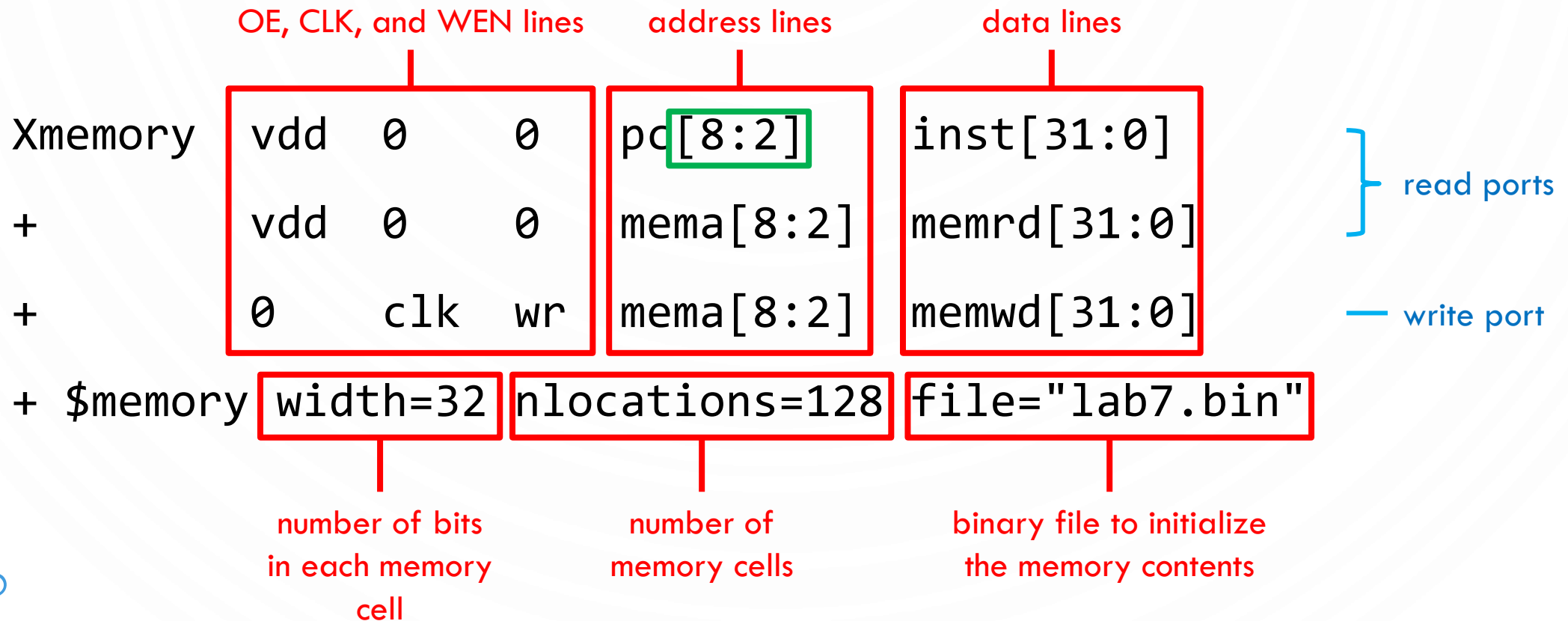
- The order of ra and rc are swapped for the ST instruction – *this is by design!*

```
INSTRUCTIONS (SUPERVISOR MODE)
000:601F019C      LD(R31,412,R0)
004:C0210004 LOOP:  ADDC(R1,4,R1)
008:6041019C      LD(R1,412,R2)
00C:80631000      ADD(R3,R2,R3)
010:C4000001      SUBC(R0,1,R0)
014:7BE0FFFB      BNE(R0,LOOP)
018:647F01B4      ST(R3,436,R31)
01C:00000000      HALT()
```


The JSim memory device

```
Xmemory  vdd  0    0    pc[8:2]    inst[31:0]
+        vdd  0    0    mema[8:2]  memrd[31:0]
+        0    clk  wr    mema[8:2]  memwd[31:0]
+ $memory width=32 nlocations=128 file="lab7.bin"
```

The JSim memory device



OE, CLK, and WEN

- OE = output enable
 - Should be ON for a read port, OFF for a write port
- CLK = a clock signal
 - Valid for write ports only (should be always OFF for read ports)
 - Tells the device to write (on the clock's rising edge)
- WEN = write enable
 - Valid for write ports only (should be always OFF for read ports)
 - Should only be ON when you want to write what is currently being emitted by the data lines

We'll use a memory device for our registers too

```
Xregisters vdd 0 0 ra1_[4:0] rd1_[31:0]
+         vdd 0 0 ra2_[4:0] rd2_[31:0]
+         0 clk werf wa[4:0] wd[31:0]
+ $memory width=32 nlocations=32 contents=(
+ 0 0 0 0 0 0 0 0
+ 0 0 0 0 0 0 0 0
+ 0 0 0 0 0 0 0 0
+ 0 0 0 0 0 0 0 0
+ )
```

Pay special attention to the WEN signal

- Both memory and register blocks have a “Write ENable” (WEN) flag:
 - For the memory block, we will call it Wr
 - For the register block, we will call it WERF (Write Enable to Register File)
- These should be set to ON only when you want to write
 - Failure to turn these signals off can be disastrous for your processor!

Quirk Alert: Beta's R31 register

- Register 31 must be “always zero”
- You must add circuitry to disable writing to the register file if $wa[4:0] = 0b11111$ (31)
 - Acceptable solution: Instead of feeding WERF directly into Xregisters, feed an intermediate signal WERF_ that is turned on only if:
 - ✓ WERF is on
 - ✓ $wa[4:0]$ is not equal to $0b11111$

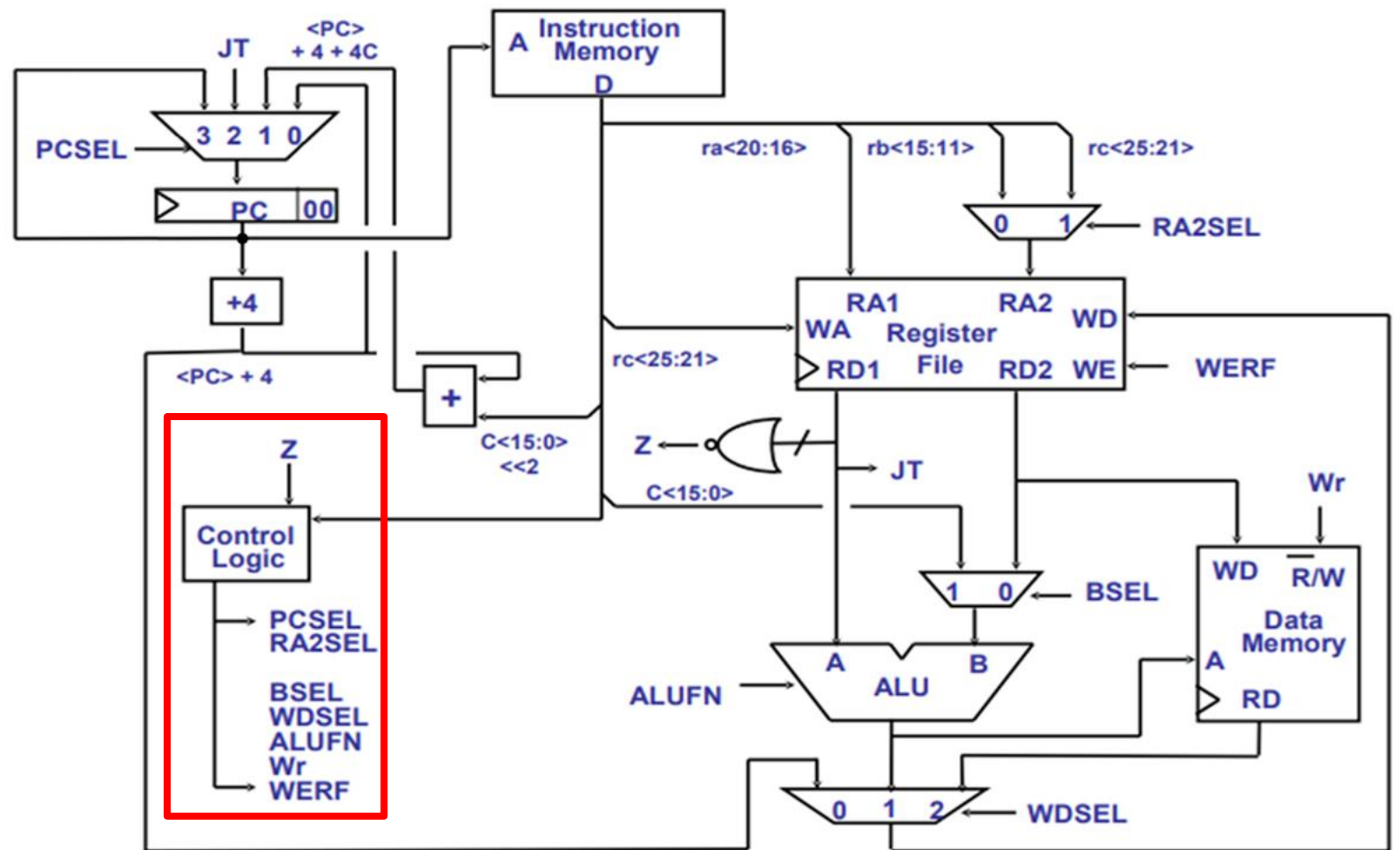
So, when do we actually turn on Wr and WERF?

- This is the function of a control logic circuit
- In fact, control logic also determines other things:
 - What ALU operation to perform (ALUFN)
 - From what address will the next instruction be fetched from (PCSEL)
 - ... and so on...

The Beta Datapath

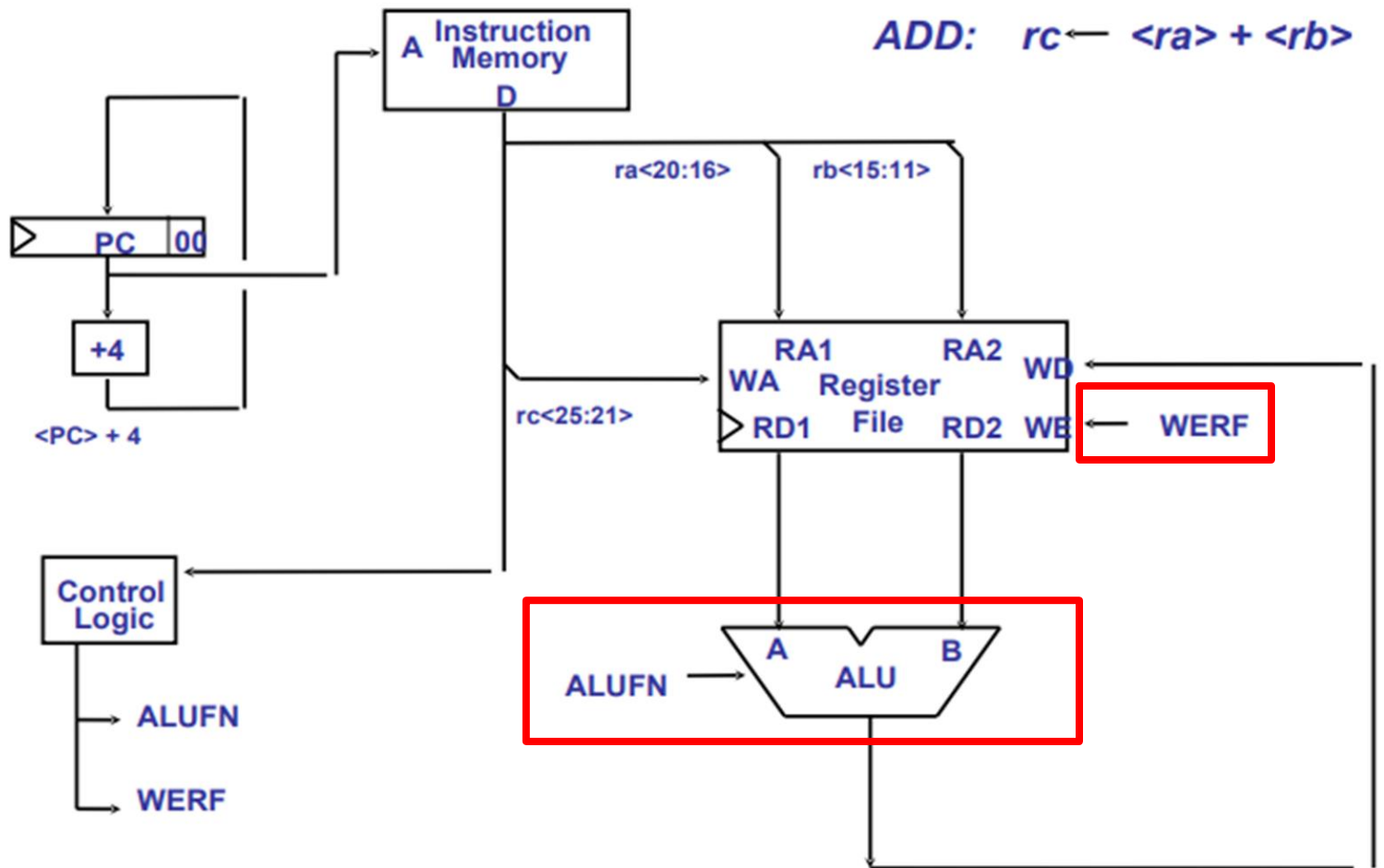
- This is the same datapath from CSCI 50.01, but simplified:

- No PCSEL 4
- No ASEL
- No WASEL
- No IRQ



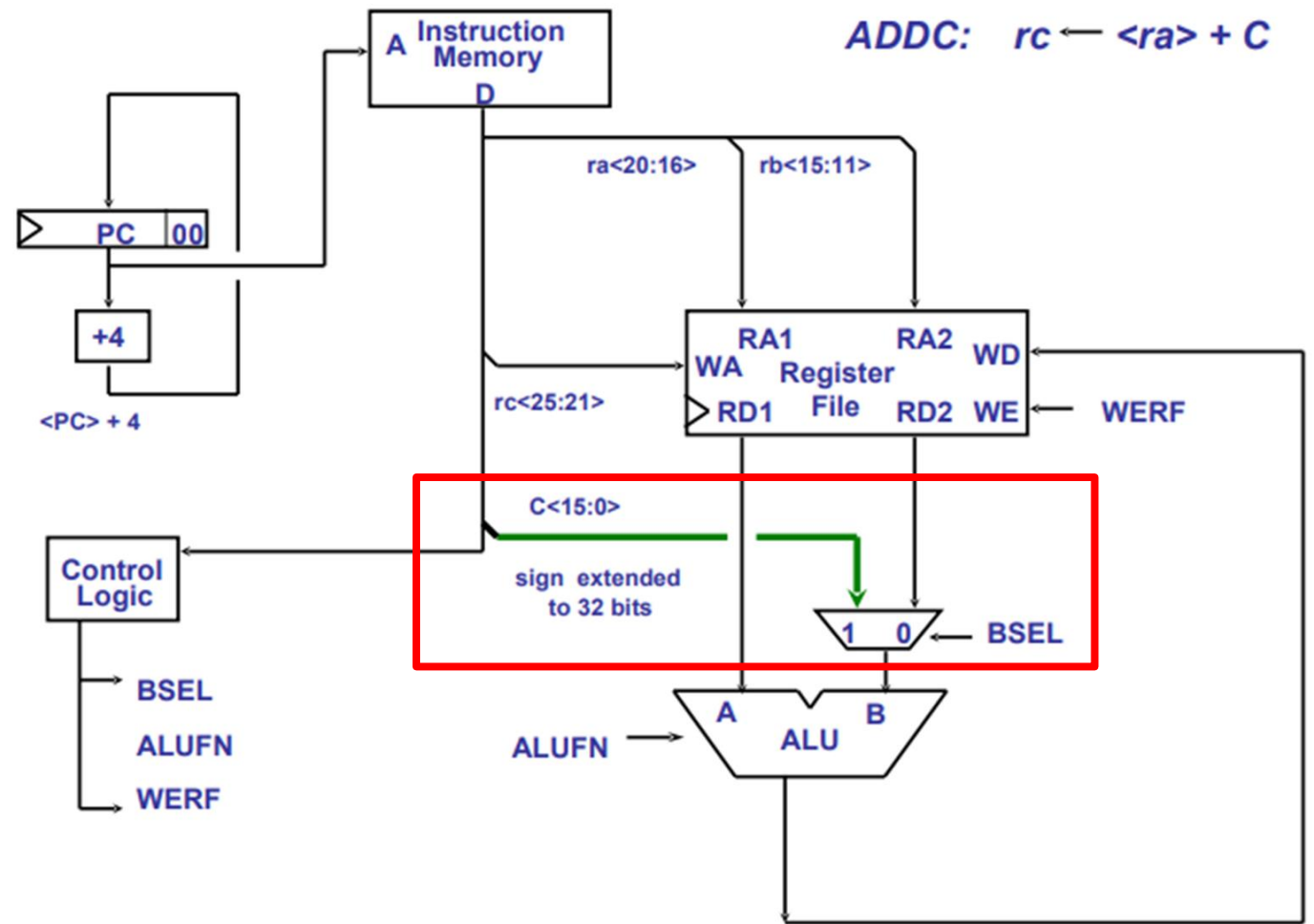
Review: ALU Register-Register Instructions (e.g., ADD, SUB...)

- PCSEL = 00
- RA2SEL = 0
- BSEL = 0
- **WDSEL = 01**
- **ALUFN = depends**
- $W_r = 0$
- **WERF = 1**



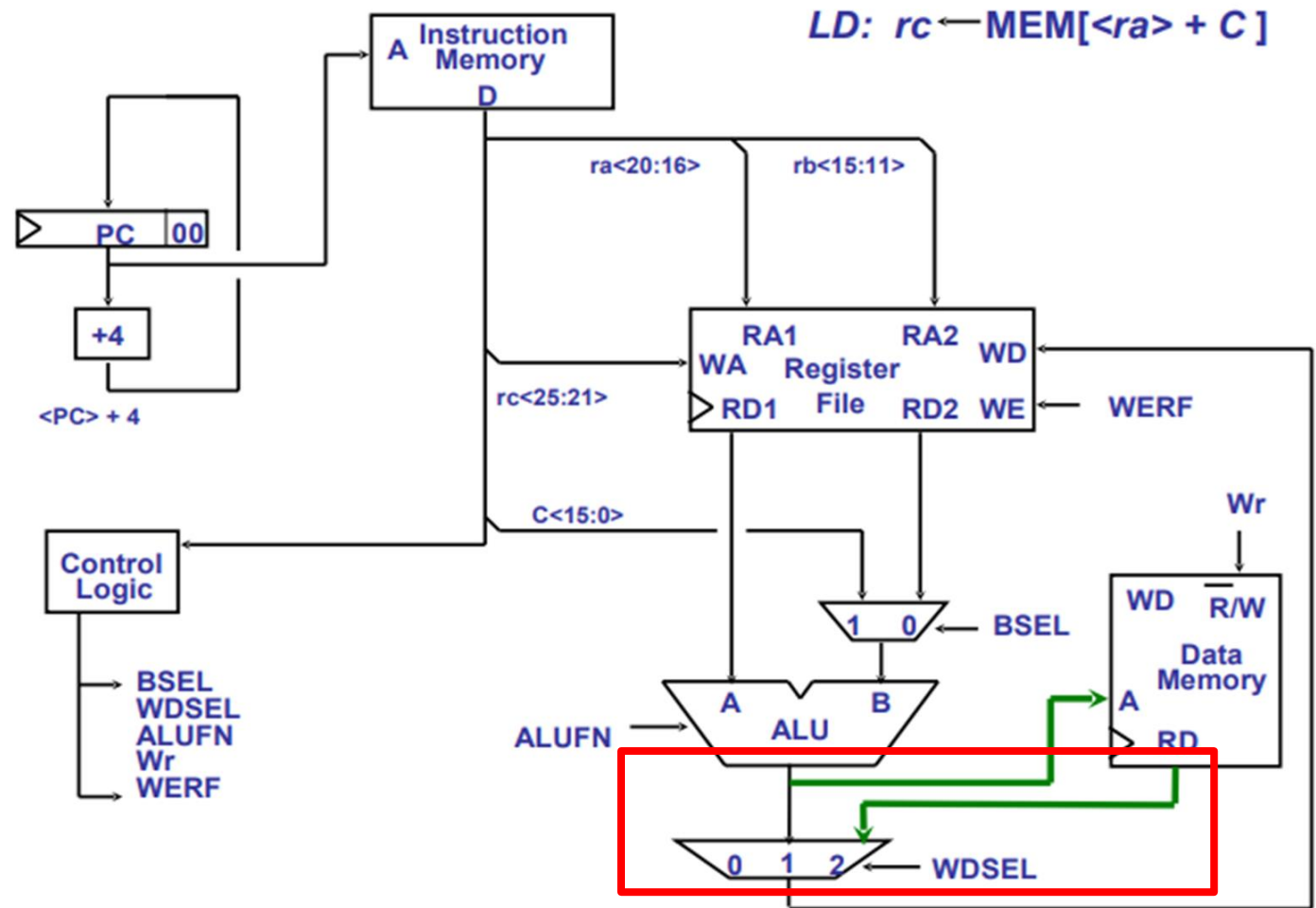
Review: ALU Register-Constant Instructions (e.g., ADDC, SUBC...)

- PCSEL = 00
- RA2SEL = 0
- **BSEL = 1**
- WDSEL = 01
- ALUFN = depends
- $W_r = 0$
- WERF = 1



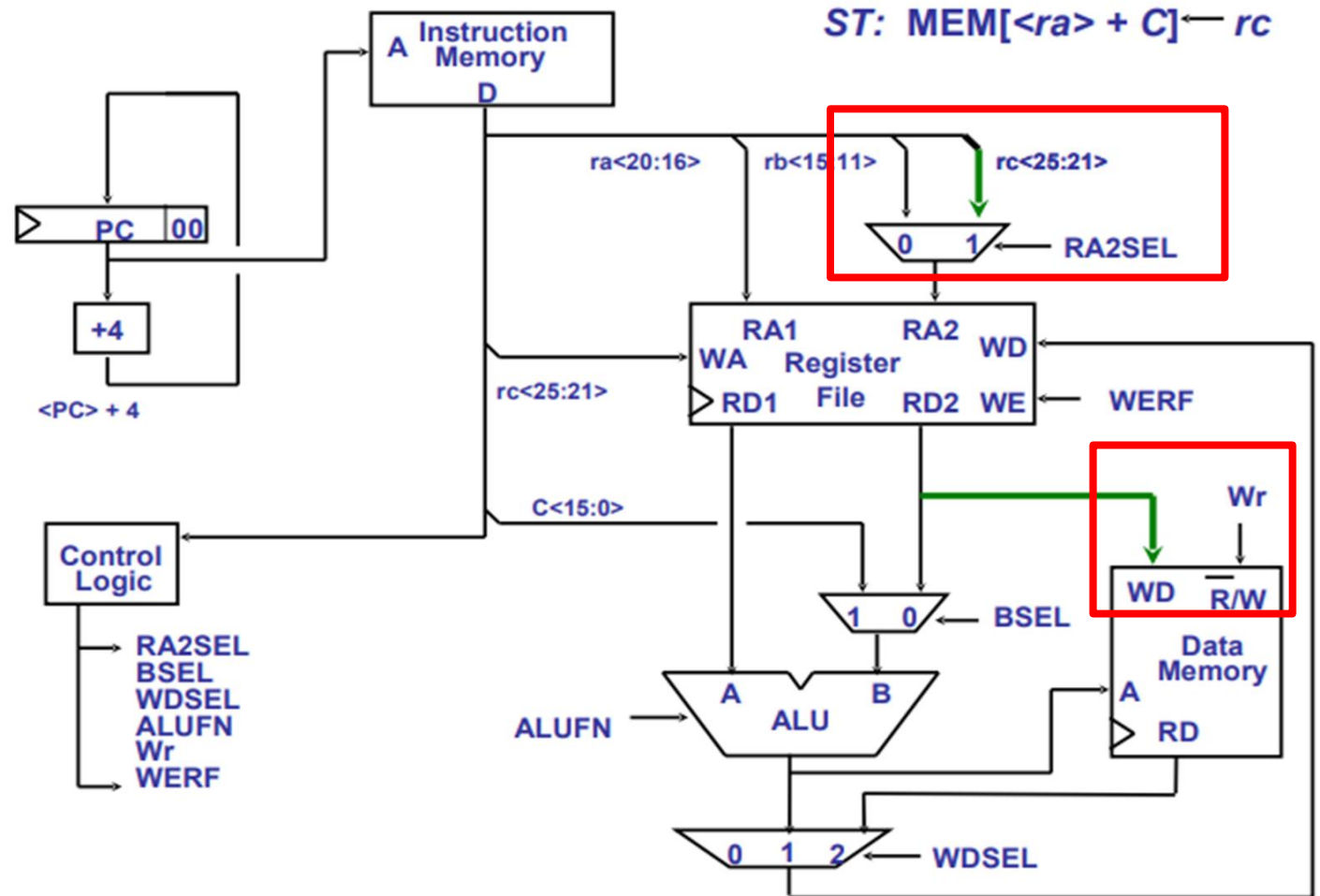
Review: LD Instruction

- PCSEL = 00
- RA2SEL = 0
- BSEL = 1
- **WDSEL = 10**
- **ALUFN = add**
- $W_r = 0$
- WERF = 1



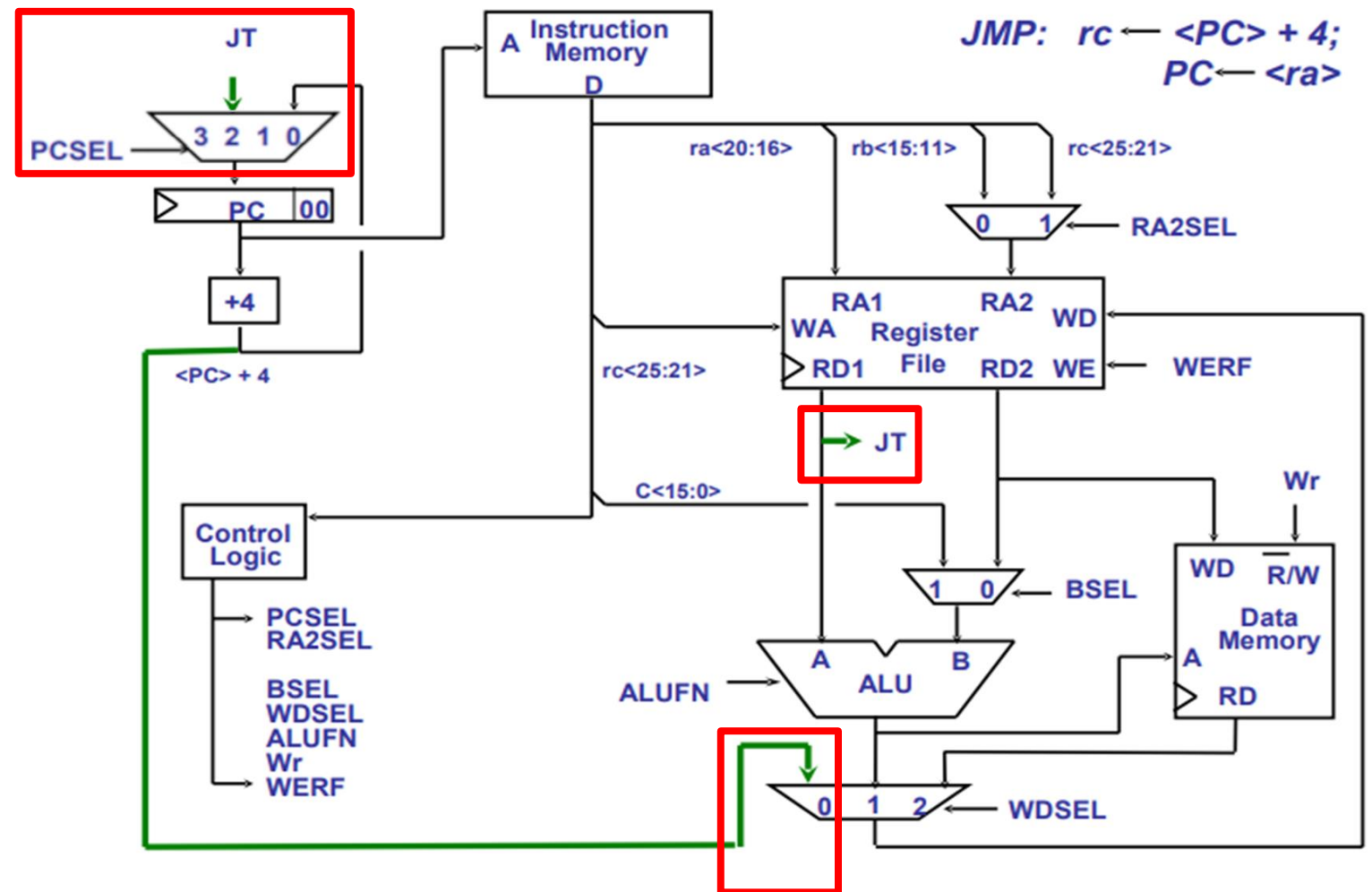
Review: ST Instruction

- PCSEL = 00
- RA2SEL = 1
- BSEL = 1
- WDSEL = dontcare
- ALUFN = add
- $W_r = 1$
- WERF = 0



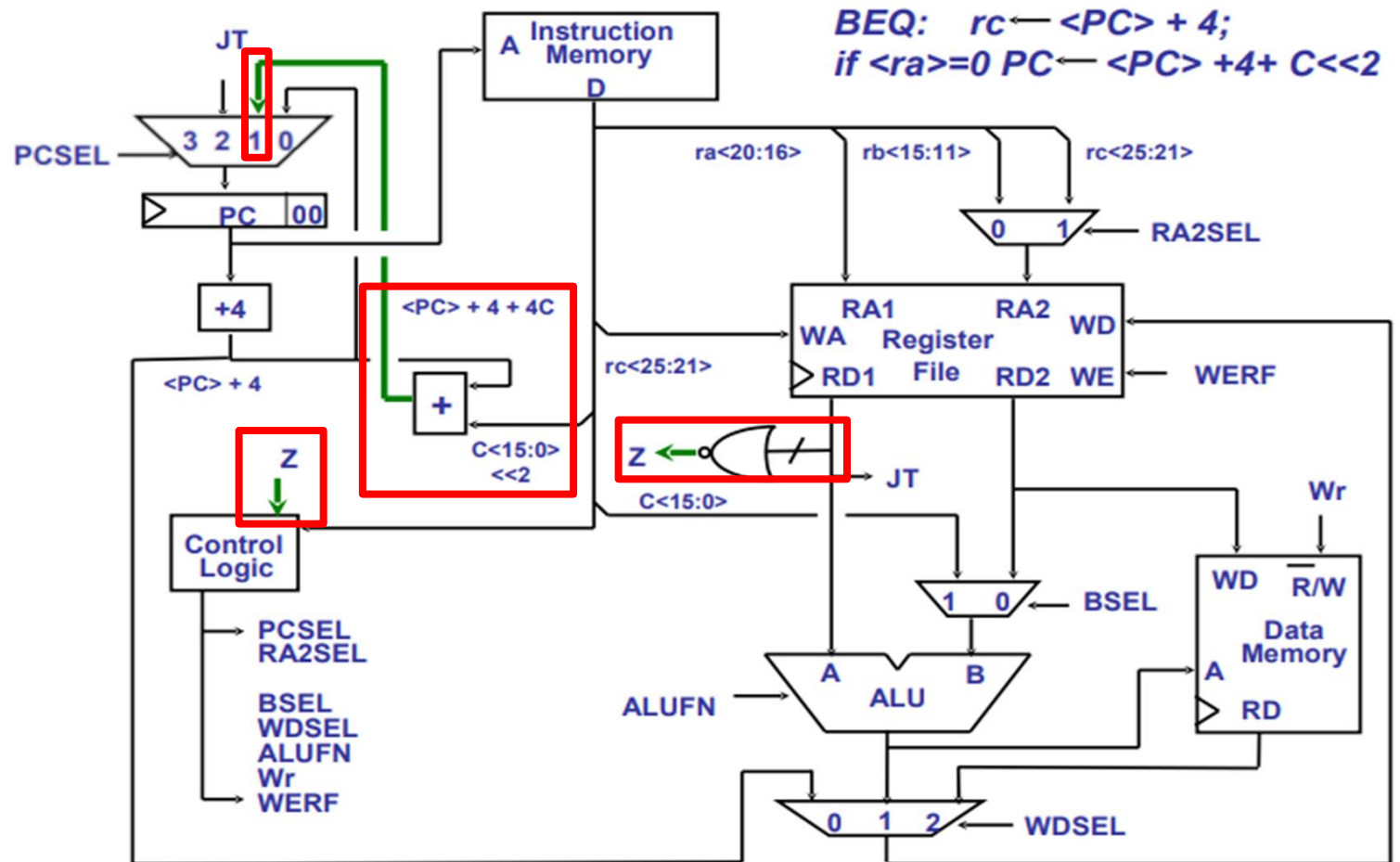
Review: JMP Instruction

- **PCSEL = 10**
- RA2SEL = dontcare
- BSEL = dontcare
- **WDSEL = 00**
- ALUFN = dontcare
- **Wr = 0**
- **WERF = 1**



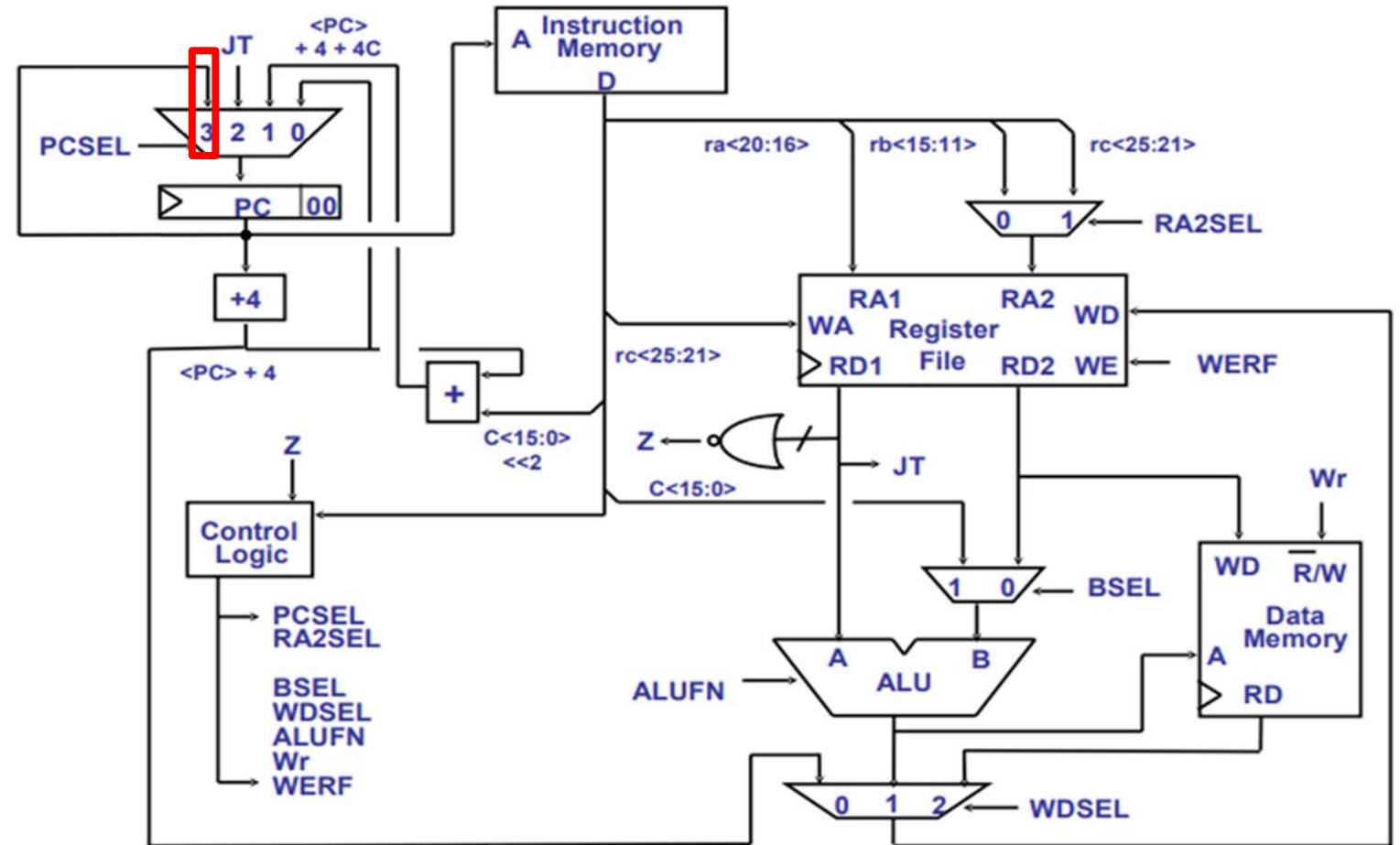
Review: BEQ/BNE Instruction

- PCSEL = depends on Z
- RA2SEL = dontcare
- BSEL = dontcare
- WDSEL = 00
- ALUFN = dontcare
- $W_r = 0$
- WERF = 1



HALT Instruction

- **PCSEL = 11**
- RA2SEL = dontcare
- BSEL = dontcare
- WDSEL = dontcare
- ALUFN = dontcare
- **Wr = 0**
- **WERF = 0**



So how does the actual Control Logic circuit look like?

- Inputs:
 - The current instruction's opcode
 - Z signal (true when `rd1_[31:0]` is zero)
- Outputs:
 - All relevant control logic signals (14 bits)
- Since the input-output combinations are static, we can use a lookup table
 - Read-only memory (ROM)
 - Memory addresses = inputs of your control logic
 - Memory data = outputs of your control logic

The Control Logic ROM

```
Xcontrol  vdd  0  0  z opcode[5:0]  control[13:0]
+ $memory width=14 nlocations=128 contents=(
+   0b11000000000000 0b11000000000000 ...
*   fill up this table with the correct control signals
*   (the order of the bits from left to right is PCSEL,
*   RA2SEL, BSEL, WDSEL, ALUFN, Wr, and WERF)
+ )
```

The OPCODE table

For our exercise, treat all the red cells as invalid/HALT

5:3 \ 2:0	000	001	010	011	100	101	110	111
000	HALT							
001								
010								
011	LD	ST		JMP		BEQ	BNE	LDR*
100	ADD	SUB	MUL*	DIV*	CMPEQ	CMPLT	CMPLE	
101	AND	OR	XOR		SHL	SHR	SRA	
110	ADDC	SUBC	MULC*	DIVC*	CMPEQC	CMPLTC	CMPLEC	
111	ANDC	ORC	XORC		SHLC	SHRC	SRAC	

Lab #8 Specifications (page 1 of 2)

1. Add circuitry to lab8.jsim (near the Xregisters circuit block) to handle the special case for register 31 (which must always be zero)
2. Complete the Xcontrol circuit block in lab8.jsim by filling in the correct values for your control ROM table
3. Demonstrate that your new Xcontrol circuit works with the .bin output of one of your Lab 7 programs, via JSim simulation
 - The plotting commands in lab8.jsim also show the current values of the control signals, so that you can check if they are correct per instruction

Lab #8 Specifications (page 2 of 2)

4. Answer the following guide questions:

- a. What is the true purpose of the additional Z input (on top of the instruction opcode inputs) in the Xcontrol block?
- b. What is the true purpose of setting the invalid instructions' control logic output to the constant 0b11000000000000 ?
- c. Does your Lab 7 program (that you yourself provided) represent a good number of opcodes that your control logic supports? What's the functional coverage? (i.e., which opcodes did you implement in your control logic table, and which ones are actually exercised by your Lab 7 program?)

Lab #8 Format

- All submissions done via Canvas
 - Just your main .jsim file, but rename the file's extension to .jsim.txt
 - e.g., ourLab.jsim becomes ourLab.jsim.txt
 - In the first few line comments, put your names in “Last Name, First Name” form
 - One line per name, sorted by last name
 - Your submission should only reference 8clocks.jsim, nominal.jsim and stdcell.jsim (provided in jsim.zip) via the .include command -- do not physically include these other .jsim files in your submission!
- Also submit a Certificate of Authorship (as a .pdf with your e-signatures)
- You may (optionally) send a private Canvas message for peer evaluations